1. Greedy Rod-Cutting Counterexample

Instance: $n = 4$, $P = [1, 5, 8, 9]$.

i	1	2	3	4
$P[i]$	1	5	8	9
$U[i] = P[i]/i$	1	5/2	8/3	9/4

Greedy: $U^* = U[3] = 8/3$, so $i^* = 3$. Cut $\lfloor 4/3 \rfloor = 1$ piece of length 3, leftover $4 - 3 = 1$.

$$\text{Revenue} = P[3] + P[1] = 8 + 1 = 9$$

All partitions of $n = 4$:

$$\begin{aligned}
(4) &: P[4] = 9 \\
(3,\,1) &: P[3] + P[1] = 8 + 1 = 9 \\
(2,\,2) &: 2 \cdot P[2] = 2 \cdot 5 = 10 \\
(2,\,1,\,1) &: P[2] + 2 \cdot P[1] = 5 + 2 = 7 \\
(1,\,1,\,1,\,1) &: 4 \cdot P[1] = 4
\end{aligned}$$

Optimal: $(2, 2)$, revenue $= 10$.

$$\boxed{\text{Greedy} = 9 < 10 = \text{OPT}}$$

Greedy fails: $3 \nmid 4$, so the leftover piece of length 1 has $U[1] = 1 < 8/3$.

2. Minimum Subset Sums

Claim: Olivia's algorithm is **not** optimal.

Counterexample: $n = 2$, $A = [1, 100]$, $W = 100$.

$$\begin{aligned}
&\text{Sorted nondecreasing:} \quad A = [1, 100] \\
&S[1] = 1 < 100, \quad S[2] = 101 \geq 100 \\
&\implies i^* = 2
\end{aligned}$$

$$\begin{aligned}
\text{Olivia:} \quad & B = [1, 1], \quad \textstyle\sum B[i] = 2 \\
\text{Optimal:} \quad & B = [0, 1], \quad A[2] = 100 \geq W, \quad \textstyle\sum B[i] = 1
\end{aligned}$$

$$\boxed{\text{Olivia's algorithm is not optimal} \quad (2 > 1)}$$

Sorting nondecreasing selects the *smallest* elements first; minimizing count requires selecting the *largest* first.

Runtime:

$$\begin{aligned}&\text{Sort:} \quad O(n \log n)\\ &\text{Prefix sums:} \quad O(n)\\ &\text{Binary search for } i^*\text{:} \quad O(\log n)\end{aligned}$$

$$\boxed{O(n \log n)}$$

3. Subset Sum with Conflict Pairs

Let $V = \bigcup_{(x,y)\in L}\{x, y\}$ and $m = |V| \le 2k$. Number the elements of V in increasing order as $v_0, v_1, \ldots, v_{m-1}$, with pos: $V \to \{0, \ldots, m-1\}$. For each $v_j \in V$, let $N_j \subseteq V$ be its *conflict neighborhood*:

$$N_j = \{\, v_\ell : (v_j, v_\ell) \in L \text{ or } (v_\ell, v_j) \in L \,\}$$

State: $DP[i][w][S]$ = maximum sum from items $\{1, \ldots, i\}$ with total $\le w$, where $S \subseteq V$ is the set of selected conflict items.

Base: $DP[0][w][\emptyset] = 0$ for all w; $\quad DP[0][w][S] = -\infty$ for $S \neq \emptyset$.

Recurrence: Set $DP[i][w][S] \longleftarrow DP[i-1][w][S]$ (skip item i), then update:

$$\begin{aligned}&\text{If } i \notin V \text{ and } w \ge A[i]\text{:}\\ &\qquad DP[i][w][S] \longleftarrow \max\big(DP[i][w][S],\ A[i] + DP[i-1][w-A[i]][S]\big)\\ &\text{If } i = v_j \in V,\ v_j \in S,\ S \cap N_j = \emptyset,\ w \ge A[i]\text{:}\\ &\qquad DP[i][w][S] \longleftarrow \max\big(DP[i][w][S],\ A[i] + DP[i-1][w-A[i]][S \setminus \{v_j\}]\big)\end{aligned}$$

Answer: $\max\limits_{S\subseteq V} DP[n][W][S]$.

Implementation: Subsets $S \subseteq V$ are stored as m-bit integers s (bit j is 1 iff $v_j \in S$), so $v_j \in S$ becomes $s \,\&\, 2^j \neq 0$, $S \cap N_j = \emptyset$ becomes $s \,\&\, N[j] = 0$, and $S \setminus \{v_j\}$ becomes $s \oplus 2^j$. The arrays prev$[w][s]$ and curr$[w][s]$ store $DP[i-1][w][S]$ and $DP[i][w][S]$.

```
begin
    ConflictSubsetSum(A[1..n], W, L[1..k])
    V ⟵ ⋃_{(x,y)∈L} {x, y};   m ⟵ |V|
    Map V to positions {0, ..., m−1} via pos
    for j ⟵ 0 to m − 1 do
        N[j] ⟵ 0
    for each (x, y) ∈ L do
        N[pos(x)] ⟵ N[pos(x)] | 2^pos(y)
        N[pos(y)] ⟵ N[pos(y)] | 2^pos(x)
    for w ⟵ 0 to W do
        for s ⟵ 0 to 2^m − 1 do
            prev[w][s] ⟵ −∞
        prev[w][0] ⟵ 0
    for i ⟵ 1 to n do
        curr ⟵ prev
        for w ⟵ A[i] to W do
            for s ⟵ 0 to 2^m − 1 do
                if i ∉ V then
                    curr[w][s] ⟵ max(curr[w][s], A[i] + prev[w−A[i]][s])
                else if s & 2^pos(i) ≠ 0 and s & N[pos(i)] = 0 then
                    j ⟵ pos(i)
                    curr[w][s] ⟵ max(curr[w][s], A[i] + prev[w−A[i]][s ⊕ 2^j])
        prev ⟵ curr
    return max_s prev[W][s]
```

Runtime:

Preprocessing: $O(k)$
DP: $n \times W \times 2^m$ states, $O(1)$ per state
$m = |V| \leq 2k$

$$\boxed{O(n \cdot W \cdot 4^k)}$$

4. Longest Palindrome Subsequence

State: $DP[i][j]$ = length of the longest palindrome subsequence (LPS) of $S[i..j]$.

Base: $DP[i][i] = 1$; $DP[i][j] = 0$ for $i > j$.

Recurrence:

$$DP[i][j] = \begin{cases} DP[i{+}1][j{-}1] + 2 & \text{if } S[i] = S[j] \\ \max\big(DP[i{+}1][j],\ DP[i][j{-}1]\big) & \text{if } S[i] \neq S[j] \end{cases}$$

```
begin
    LPS(S[1..n])
    for i ⟵ 1 to n do
        DP[i][i] ⟵ 1
    for ℓ ⟵ 2 to n do
        for i ⟵ 1 to n − ℓ + 1 do
            j ⟵ i + ℓ − 1
            if S[i] = S[j] then
                DP[i][j] ⟵ DP[i+1][j−1] + 2
            else
                DP[i][j] ⟵
                  max(DP[i+1][j], DP[i][j−1])
    return Reconstruct(1, n)
```

```
begin
    Reconstruct(i, j)
    if i > j then
        return ε
    if i = j then
        return S[i]
    if S[i] = S[j] then
        return S[i] ·
          Reconstruct(i+1, j−1) · S[j]
    if DP[i+1][j] ≥ DP[i][j−1] then
        return Reconstruct(i+1, j)
    return Reconstruct(i, j−1)
```

Example: $S =$ `character` ($n = 9$: c h a r a c t e r).

$$\begin{aligned} &DP[3][5] = 3 \quad (S[3..5] = \text{“ara”},\ S[3] = S[5] = \text{a} \implies DP[4][4] + 2 = 3) \\ &DP[2][5] = 3 \quad (S[2] \neq S[5] \implies \max(DP[3][5],\ DP[2][4]) = 3) \\ &DP[1][6] = 5 \quad (S[1] = S[6] = \text{c} \implies DP[2][5] + 2 = 5) \\ &DP[1][9] = 5 \quad (\text{propagated through } DP[1][7], DP[1][8]) \end{aligned}$$

Reconstruction: $DP[1][9] \to DP[1][6]$: select **c**; $\to DP[3][5]$: select **a**; $\to DP[4][4]$: select **r**.

`carac` (length 5)

Runtime:

Fill: $O(n^2)$ subproblems, $O(1)$ each
Reconstruct: $O(n)$ steps

$O(n^2)$ time, $O(n^2)$ space